Institución Universitaria Pascual Bravo

Base de Datos I

Tarea 3: Implementación del modelo físico en PostgreSQL (DDL)

Red social estudiantil "Pascualina"

Grupo 9

Integrantes:

Andry Zharick Mariño Moreno

Alejandro Gallego Velásquez

2026

Introducción

En la Tarea 2 hicimos el modelo lógico de la red social Pascualina, con 16 tablas. En esta tarea lo construimos de verdad en un programa de bases de datos (un SGBD) escribiendo un script DDL, o sea, las instrucciones como CREATE TABLE que crean las tablas, sus llaves y sus reglas.

1. SGBD elegido y justificación

Elegimos PostgreSQL y lo trabajamos con pgAdmin 4, que es el programa con ventanas para usarlo: ahí se escribe el script, se ven las tablas y se genera el diagrama. Lo comparamos con tres opciones más:

SGBD	Ventajas	Desventajas
PostgreSQL (elegido)	Es gratis. Siempre hace cumplir las reglas y las llaves entre tablas. Aguanta muchos usuarios a la vez. pgAdmin dibuja el diagrama.	Al principio es un poco más difícil de instalar y de aprender.
MySQL	Es gratis y muy conocido. Es fácil de usar y tiene su propio programa con ventanas (Workbench).	Hace cumplir menos las reglas (en versiones viejas ignora algunas). Es de Oracle y algunas funciones son de pago.
SQLite	No necesita instalar un servidor, es un solo archivo. Es muy liviana.	Solo deja escribir a un usuario a la vez, así que no sirve para muchos usuarios conectados. Las llaves entre tablas vienen apagadas por defecto.
SQL Server Express	Es muy completo y tiene un buen programa con ventanas.	La versión gratis tiene límites de tamaño y de memoria; la completa es de pago. Su programa solo funciona en Windows.

Por eso escogimos PostgreSQL:

- Es gratis, así que los dos pudimos instalarlo sin pagar nada.
- Nuestro modelo tiene 22 relaciones entre tablas, y PostgreSQL siempre hace cumplir esas relaciones y las reglas.
- Una red social tiene muchos usuarios a la vez, y PostgreSQL está hecho para eso.
- pgAdmin genera el diagrama directo desde la base de datos, y el material del curso (Canepa, 2016; Hsu y Obe, 2015) es de PostgreSQL.

2. Descripción del modelo físico

El modelo físico es el modelo lógico de la Tarea 2 escrito en SQL para PostgreSQL. Dejamos las mismas 16 tablas, los mismos campos y las mismas 22 relaciones. Solo se cambió el nombre de un campo: la fecha de reaccion pasó a llamarse fecha_reaccion para que se entienda mejor.

2.1 Reglas para los nombres

- Todo en minúscula y con guion bajo, sin tildes ni espacios (por ejemplo, estudiante_habilidad).
- Las tablas en singular (estudiante y no estudiantes).
- Llave primaria: id_ más el nombre de la tabla. Llave foránea: el mismo nombre de la llave a la que apunta, o un nombre que diga el papel (id_seguidor, id_mentor, creador_id).

- Las restricciones tienen nombre con prefijo: pk_ (llave primaria), fk_ (llave foránea), uk_ (única) y ck_ (regla de valores).

2.2 Cómo pasamos del lógico al físico

En el modelo lógico	En PostgreSQL
Tabla	CREATE TABLE, con el nombre en minúscula
Llave primaria (INT)	SERIAL, que es un número que se llena solo, más la restricción pk_
Llave foránea	Restricción fk_ con REFERENCES, ON DELETE y ON UPDATE
Relación muchos a muchos	Tabla intermedia con dos llaves foráneas y UNIQUE para que no se repita
DATETIME	TIMESTAMP, porque PostgreSQL no tiene DATETIME
Dato obligatorio / valor automático	NOT NULL / DEFAULT (por ejemplo, la fecha de hoy)

2.3 Inventario de tablas

#	Tabla	Qué guarda
1	estudiante	Cada usuario de la red social
2	perfil	Foto, biografía y datos del perfil
3	habilidad	Catálogo de habilidades
4	interes	Catálogo de intereses
5	estudiante_habilidad	Habilidades de cada estudiante y su nivel
6	estudiante_interes	Intereses de cada estudiante
7	grupo	Grupos de estudio, equipos y clubes
8	miembro_grupo	Qué estudiante está en qué grupo
9	lugar	Sitios donde se hacen los eventos
10	evento	Reuniones, talleres y actividades
11	asistente_evento	Quién se inscribió a qué evento
12	publicacion	Preguntas, recursos, noticias y memes
13	comentario	Comentarios de las publicaciones
14	reaccion	Reacciones a las publicaciones
15	seguir	Quién sigue a quién
16	mentoria	Mentorías entre estudiantes

3. El script DDL y su estructura

El script se llama Script_Tarea3_Grupo9.sql y está en el repositorio de GitHub, en la carpeta Tarea3. Se ejecuta completo de una vez en el Query Tool de pgAdmin 4. Su estructura es esta:

- Encabezado: nombre del proyecto, integrantes y las reglas de nombres.
- Limpieza: DROP TABLE IF EXISTS para borrar las tablas si ya existían, empezando por las hijas. Así el script se puede ejecutar varias veces.
- Las 16 tablas, una por una con su CREATE TABLE: primero las columnas y después las restricciones (llave primaria, única, regla de valores y llaves foráneas).
- Verificación: una consulta que lista las tablas creadas.

Las tablas se crean en un orden que respeta las dependencias: primero las que no apuntan a nadie (estudiante, habilidad, interes, lugar) y después las que dependen de ellas, porque una llave foránea no puede apuntar a una tabla que todavía no existe. Cada línea tiene un comentario con -- que explica para qué sirve. Este es un ejemplo con la tabla estudiante:

```
CREATE TABLE estudiante (  -- Tabla de los usuarios de la red social
    id_estudiante SERIAL,  -- Número que identifica al estudiante (automático)
    nombre VARCHAR(50) NOT NULL,  -- Nombre (obligatorio)
    apellido VARCHAR(50) NOT NULL,  -- Apellido (obligatorio)
    correo VARCHAR(100) NOT NULL,  -- Correo (obligatorio)
    password_hash VARCHAR(255) NOT NULL,  -- Contraseña cifrada (obligatoria)
    programa_academico VARCHAR(80) NOT NULL,  -- Carrera (obligatoria)
    fecha_registro DATE NOT NULL DEFAULT CURRENT_DATE,  -- Fecha de registro (hoy por defecto)
    ultima_actividad TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  -- Última vez que entró
    (ahora por defecto)
    estado VARCHAR(20) NOT NULL DEFAULT 'activo',  -- Estado de la cuenta ('activo' por
    defecto)
    CONSTRAINT pk_estudiante PRIMARY KEY (id_estudiante),  -- Llave primaria: no se repite ni
    queda vacía
    CONSTRAINT uk_estudiante_correo UNIQUE (correo),  -- No puede haber dos cuentas con el
    mismo correo
    CONSTRAINT ck_estudiante_estado CHECK (estado IN ('activo', 'inactivo'))  -- Solo permite
    'activo' o 'inactivo'
);  -- Fin de la tabla estudiante
```

4. Descripción de cada tabla

Para cada tabla se muestran los campos con su tipo de dato y tamaño, la clave primaria (PK), las llaves foráneas (FK) y las llaves únicas (UQ), y por qué elegimos la clave primaria y los tipos de dato principales.

4.1 Tabla estudiante

Guarda a cada usuario de la red social.

Campo	Tipo y tamaño	Clave	Para qué sirve
id_estudiante	SERIAL (INTEGER)	PK	Identificador del estudiante
nombre	VARCHAR(50)		Nombre
apellido	VARCHAR(50)		Apellido
correo	VARCHAR(100)	UQ	Correo institucional, no se repite
password_hash	VARCHAR(255)		Contraseña guardada cifrada
programa_academico	VARCHAR(80)		Carrera que estudia
fecha_registro	DATE		Día en que se registró
ultima_actividad	TIMESTAMP		Última vez que entró
estado	VARCHAR(20)		Cuenta activa o inactiva

Clave primaria: id_estudiante. Se usó un número automático y no el correo porque el correo se puede cambiar y un número es más rápido para buscar y conectar tablas.

Tipos de dato: VARCHAR en nombre, apellido y correo porque son textos cortos con un límite. INTEGER (SERIAL) en el id porque alcanza para millones de estudiantes.

4.2 Tabla perfil

Guarda lo que el estudiante muestra de sí mismo.

Campo	Tipo y tamaño	Clave	Para qué sirve
id_perfil	SERIAL (INTEGER)	PK	Identificador del perfil
id_estudiante	INTEGER	FK, UQ	Estudiante dueño del perfil
foto	VARCHAR(255)		Link de la foto
biografia	TEXT		Texto libre sobre el estudiante
semestre	INTEGER		Semestre que cursa
area_estudio	VARCHAR(80)		Área de estudio
ciudad	VARCHAR(60)		Ciudad donde vive
fecha_actualizacion	DATE		Última vez que editó el perfil

Clave primaria: id_perfil, número automático. id_estudiante es UNIQUE para que cada estudiante tenga un solo perfil.

Tipos de dato: TEXT en biografia porque no tiene un largo fijo. INTEGER en id_estudiante para que sea igual al id al que apunta.

4.3 Tabla habilidad

Catálogo de habilidades (Python, liderazgo, fútbol...).

Campo	Tipo y tamaño	Clave	Para qué sirve
id_habilidad	SERIAL (INTEGER)	PK	Identificador de la habilidad
nombre	VARCHAR(60)		Nombre de la habilidad
categoria	VARCHAR(40)		Técnica, blanda, deportiva

Clave primaria: id_habilidad, número automático. Los nombres se pueden corregir, por eso no sirven de llave.

Tipos de dato: VARCHAR porque son textos cortos.

4.4 Tabla interes

Catálogo de intereses (videojuegos, programación competitiva...).

Campo	Tipo y tamaño	Clave	Para qué sirve
id_interes	SERIAL (INTEGER)	PK	Identificador del interés
nombre	VARCHAR(60)		Nombre del interés
categoria	VARCHAR(40)		Académico, deportivo, ocio

Clave primaria: id_interes, número automático.

Tipos de dato: VARCHAR porque son textos cortos.

4.5 Tabla estudiante_habilidad

Tabla intermedia: qué habilidades tiene cada estudiante y con qué nivel.

Campo	Tipo y tamaño	Clave	Para qué sirve
id_estudiante_habilidad	SERIAL (INTEGER)	PK	Identificador del registro
id_estudiante	INTEGER	FK	Estudiante que tiene la habilidad
id_habilidad	INTEGER	FK	Habilidad que tiene
nivel	VARCHAR(20)		Básico, intermedio o avanzado
fecha_agregado	DATE		Día en que la agregó

Clave primaria: id_estudiante_habilidad, número automático. Con UNIQUE sobre (id_estudiante, id_habilidad) no se repite la misma habilidad.

Tipos de dato: INTEGER en las llaves foráneas porque deben ser iguales a las llaves primarias a las que apuntan.

4.6 Tabla estudiante_interes

Tabla intermedia: qué intereses tiene cada estudiante.

Campo	Tipo y tamaño	Clave	Para qué sirve
id_estudiante_interes	SERIAL (INTEGER)	PK	Identificador del registro
id_estudiante	INTEGER	FK	Estudiante que tiene el interés
id_interes	INTEGER	FK	Interés que tiene
fecha_agregado	DATE		Día en que lo agregó

Clave primaria: id_estudiante_interes, número automático. Con UNIQUE sobre (id_estudiante, id_interes) no se repite el mismo interés.

Tipos de dato: INTEGER en las llaves foráneas porque deben ser iguales a las llaves primarias a las que apuntan.

4.7 Tabla grupo

Comunidades: grupos de estudio, equipos para hackatones y clubes.

Campo	Tipo y tamaño	Clave	Para qué sirve
id_grupo	SERIAL (INTEGER)	PK	Identificador del grupo
nombre	VARCHAR(80)		Nombre del grupo
descripcion	TEXT		De qué trata el grupo
tipo	VARCHAR(30)		Estudio, hackatón o club
fecha_creacion	DATE		Día en que se creó
estado	VARCHAR(20)		Grupo activo o cerrado
creador_id	INTEGER	FK	Estudiante que lo creó

Clave primaria: id_grupo, número automático. Los nombres de los grupos pueden parecerse o cambiar.

Tipos de dato: TEXT en descripcion porque puede ser larga. VARCHAR(80) en nombre porque es corto.

4.8 Tabla miembro_grupo

Tabla intermedia: qué estudiante está en qué grupo y con qué rol.

Campo	Tipo y tamaño	Clave	Para qué sirve
id_miembro	SERIAL (INTEGER)	PK	Identificador de la membresía
id_estudiante	INTEGER	FK	Estudiante que está en el grupo
id_grupo	INTEGER	FK	Grupo al que pertenece
fecha_ingreso	DATE		Día en que entró
rol	VARCHAR(20)		Miembro o admin

Clave primaria: id_miembro, número automático. Con UNIQUE sobre (id_estudiante, id_grupo) nadie entra dos veces al mismo grupo.

Tipos de dato: INTEGER en las llaves foráneas porque deben ser iguales a las llaves primarias a las que apuntan.

4.9 Tabla lugar

Sitios donde se hacen los eventos (biblioteca, salones...).

Campo	Tipo y tamaño	Clave	Para qué sirve
id_lugar	SERIAL (INTEGER)	PK	Identificador del lugar
nombre	VARCHAR(80)		Nombre del sitio
direccion	VARCHAR(150)		Dirección o ubicación

Clave primaria: id_lugar, número automático, porque dos sedes pueden tener salones con el mismo nombre.

Tipos de dato: VARCHAR(150) en direccion porque una dirección puede ser larga.

4.10 Tabla evento

Reuniones, talleres y actividades. Pueden venir de un grupo o de un estudiante solo.

Campo	Tipo y tamaño	Clave	Para qué sirve
id_evento	SERIAL (INTEGER)	PK	Identificador del evento
id_grupo	INTEGER	FK	Grupo que lo organiza (puede quedar vacío)
nombre	VARCHAR(80)		Nombre del evento
descripcion	TEXT		De qué trata el evento
fecha_inicio	TIMESTAMP		Fecha y hora de inicio
fecha_fin	TIMESTAMP		Fecha y hora de fin
id_lugar	INTEGER	FK	Lugar del evento
modalidad	VARCHAR(20)		Presencial o virtual
organizador_id	INTEGER	FK	Estudiante responsable

Clave primaria: id_evento, número automático, porque los nombres se repiten (por ejemplo, un taller cada semana).

Tipos de dato: TIMESTAMP en fecha_inicio y fecha_fin porque importa la hora, no solo el día.

4.11 Tabla asistente_evento

Tabla intermedia: qué estudiante se inscribió a qué evento.

Campo	Tipo y tamaño	Clave	Para qué sirve
id_asistente	SERIAL (INTEGER)	PK	Identificador de la inscripción
id_estudiante	INTEGER	FK	Estudiante que se inscribe
id_evento	INTEGER	FK	Evento al que se inscribe
estado	VARCHAR(20)		Pendiente, confirmado o cancelado
fecha_registro	DATE		Día en que se inscribió

Clave primaria: id_asistente, número automático. Con UNIQUE sobre (id_estudiante, id_evento) nadie se inscribe dos veces.

Tipos de dato: INTEGER en las llaves foráneas porque deben ser iguales a las llaves primarias a las que apuntan.

4.12 Tabla publicacion

Lo que comparten los estudiantes: preguntas, recursos, noticias o memes.

Campo	Tipo y tamaño	Clave	Para qué sirve
id_publicacion	SERIAL (INTEGER)	PK	Identificador de la publicación
id_estudiante	INTEGER	FK	Autor de la publicación
contenido	TEXT		Texto de la publicación
tipo	VARCHAR(20)		Pregunta, recurso, noticia o meme
fecha_publicacion	TIMESTAMP		Fecha y hora

Clave primaria: id_publicacion, número automático, porque dos publicaciones pueden tener el mismo texto.

Tipos de dato: TEXT en contenido porque una publicación puede ser larga. TIMESTAMP en fecha_publicacion porque importa la hora.

4.13 Tabla comentario

Comentarios que se hacen en las publicaciones.

Campo	Tipo y tamaño	Clave	Para qué sirve
id_comentario	SERIAL (INTEGER)	PK	Identificador del comentario
id_publicacion	INTEGER	FK	Publicación que se comenta
id_estudiante	INTEGER	FK	Estudiante que comenta
comentario	TEXT		Texto del comentario
fecha_comentario	TIMESTAMP		Fecha y hora

Clave primaria: id_comentario, número automático, porque pueden existir comentarios iguales.

Tipos de dato: TEXT en comentario porque cada uno tiene un largo distinto.

4.14 Tabla reaccion

Reacciones (por ejemplo, 'me gusta') a las publicaciones.

Campo	Tipo y tamaño	Clave	Para qué sirve
id_reaccion	SERIAL (INTEGER)	PK	Identificador de la reacción
id_estudiante	INTEGER	FK	Estudiante que reacciona
id_publicacion	INTEGER	FK	Publicación que recibe la reacción
tipo_reaccion	VARCHAR(20)		Me gusta, me sirve, etc.
fecha_reaccion	TIMESTAMP		Fecha y hora

Clave primaria: id_reaccion, número automático. Con UNIQUE sobre (id_estudiante, id_publicacion) hay una reacción por estudiante en cada publicación.

Tipos de dato: VARCHAR(20) en tipo_reaccion porque son pocas palabras cortas.

4.15 Tabla seguir

Quién sigue a quién. Tiene dos llaves hacia estudiante: la del que sigue y la del seguido.

Campo	Tipo y tamaño	Clave	Para qué sirve
id_seguir	SERIAL (INTEGER)	PK	Identificador del seguimiento
id_seguidor	INTEGER	FK	Estudiante que sigue
id_seguido	INTEGER	FK	Estudiante que es seguido
fecha_seguimiento	DATE		Día en que empezó a seguirlo

Clave primaria: id_seguir, número automático. Con UNIQUE sobre (id_seguidor, id_seguido) no se sigue dos veces a la misma persona.

Tipos de dato: INTEGER en las llaves foráneas porque deben ser iguales al id de estudiante.

4.16 Tabla mentoria

Mentorías: un mentor (por ejemplo, de un semestre más avanzado) acompaña a un aprendiz.

Campo	Tipo y tamaño	Clave	Para qué sirve
id_mentoria	SERIAL (INTEGER)	PK	Identificador de la mentoría
id_mentor	INTEGER	FK	Estudiante mentor
id_aprendiz	INTEGER	FK	Estudiante aprendiz
fecha_inicio	DATE		Día en que empezó
estado	VARCHAR(20)		Activa, pausada o terminada

Clave primaria: id_mentoria, número automático. No hay UNIQUE en la pareja porque una mentoría puede terminar y volver a empezar.

Tipos de dato: INTEGER en las llaves foráneas porque deben ser iguales al id de estudiante.

4.17 Por qué esos tipos de dato

- VARCHAR y TEXT: usamos VARCHAR con un tamaño máximo en los textos cortos (nombre, correo, ciudad), para que no se guarden textos exagerados. Usamos TEXT solo en los textos largos y libres (biografía, descripciones, publicaciones y comentarios).
- INTEGER y no BIGINT: INTEGER llega a más de 2.000 millones, que es muchísimo para una red social universitaria. BIGINT ocupa el doble de espacio y no lo necesitamos.
- SERIAL: es un número que se llena solo (1, 2, 3...), así los ids no se repiten ni hay que inventarlos.
- DATE y TIMESTAMP: DATE cuando solo importa el día; TIMESTAMP cuando importa la hora (inicio de un evento, hora de una publicación).

5. Relaciones entre tablas

Tenemos 22 relaciones. Cada una es una llave foránea en la tabla hija que apunta a la llave primaria de la tabla padre. Para cada una definimos qué pasa al borrar (ON DELETE) y al actualizar (ON UPDATE).

#	Tabla padre	Tabla hija	Campo de la FK	ON DELETE	ON UPDATE	Por qué
1	estudiante	perfil	id_estudiante	CASCADE	CASCADE	(1 a 1) El perfil no sirve sin su dueño.
2	estudiante	estudiante_habilidad	id_estudiante	CASCADE	CASCADE	(1 a muchos) Sin el estudiante, sus habilidades no sirven.
3	habilidad	estudiante_habilidad	id_habilidad	RESTRICT	CASCADE	(1 a muchos) Evita perder la habilidad de varios estudiantes a la vez.
4	estudiante	estudiante_interes	id_estudiante	CASCADE	CASCADE	(1 a muchos) Sin el estudiante, sus intereses no sirven.
5	interes	estudiante_interes	id_interes	RESTRICT	CASCADE	(1 a muchos) Evita perder el interés de varios estudiantes a la vez.
6	estudiante	grupo	creador_id	RESTRICT	CASCADE	(1 a muchos) Borrar al creador no debe borrar el grupo y sus miembros.
7	estudiante	miembro_grupo	id_estudiante	CASCADE	CASCADE	(1 a muchos) Una membresía sin estudiante no sirve.
8	grupo	miembro_grupo	id_grupo	CASCADE	CASCADE	(1 a muchos) Una membresía sin grupo no sirve.
9	grupo	evento	id_grupo	SET NULL	CASCADE	(1 a muchos) Un evento puede existir sin grupo, así que se conserva.
10	lugar	evento	id_lugar	RESTRICT	CASCADE	(muchos a 1) No se borra un lugar que todavía tiene eventos.
11	estudiante	evento	organizador_id	RESTRICT	CASCADE	(1 a muchos) No se deja un evento sin responsable.
12	estudiante	asistente_evento	id_estudiante	CASCADE	CASCADE	(1 a muchos) Una inscripción sin estudiante no sirve.
13	evento	asistente_evento	id_evento	CASCADE	CASCADE	(1 a muchos) Una inscripción sin evento no sirve.
14	estudiante	publicacion	id_estudiante	CASCADE	CASCADE	(1 a muchos) Las publicaciones son de su autor.
15	publicacion	comentario	id_publicacion	CASCADE	CASCADE	(1 a muchos) Un comentario sin publicación no sirve.
16	estudiante	comentario	id_estudiante	CASCADE	CASCADE	(1 a muchos) Los comentarios son de quien los escribió.

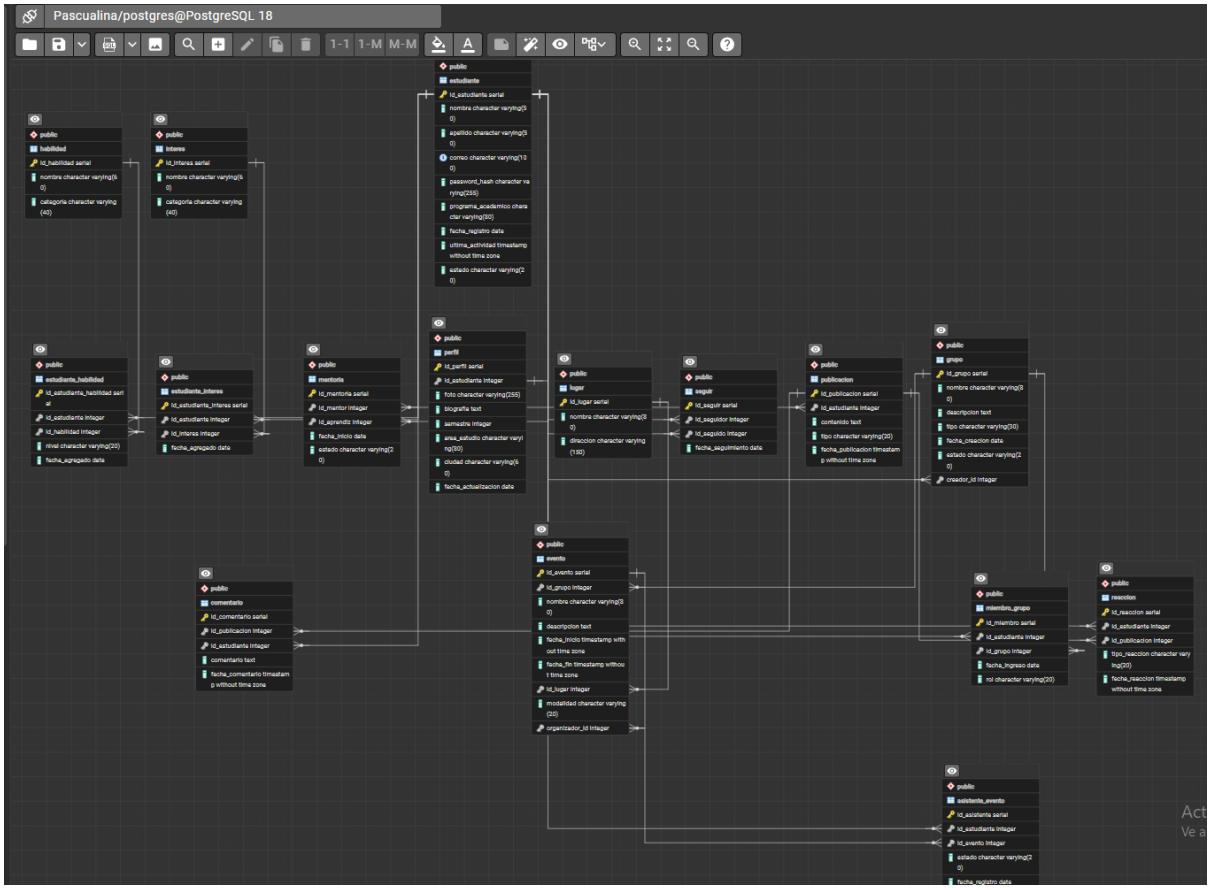
#	Tabla padre	Tabla hija	Campo de la FK	ON DELETE	ON UPDATE	Por qué
17	estudiante	reaccion	id_estudiante	CASCADE	CASCADE	(1 a muchos) Las reacciones son de quien las dio.
18	publicacion	reaccion	id_publicacion	CASCADE	CASCADE	(1 a muchos) Una reacción sin publicación no sirve.
19	estudiante	seguir	id_seguidor	CASCADE	CASCADE	(1 a muchos) Sin el estudiante, sus seguimientos no sirven.
20	estudiante	seguir	id_seguido	CASCADE	CASCADE	(1 a muchos) Nadie puede seguir a alguien que ya no existe.
21	estudiante	mentoria	id_mentor	CASCADE	CASCADE	(1 a muchos) Sin el mentor, la mentoría no existe.
22	estudiante	mentoria	id_aprendiz	CASCADE	CASCADE	(1 a muchos) Sin el aprendiz, la mentoría no existe.

Qué significa cada acción:

- CASCADE al borrar: se borran también los datos hijos. Lo usamos cuando no tienen sentido sin el padre (un comentario sin su publicación).
- RESTRICT al borrar: no deja borrar el padre mientras tenga hijos. Lo usamos para proteger datos importantes (un estudiante que creó un grupo, un lugar con eventos).
- SET NULL al borrar: el hijo se queda pero sin padre. Solo en evento.id_grupo, porque un evento puede existir sin grupo.
- CASCADE al actualizar: si algún día cambia una llave primaria, el cambio se copia solo a las llaves foráneas y no se rompe nada.
- Las relaciones de muchos a muchos (estudiante con habilidad, interés, grupo, evento, y estudiante con estudiante) se hacen con tablas intermedias que tienen dos llaves foráneas.

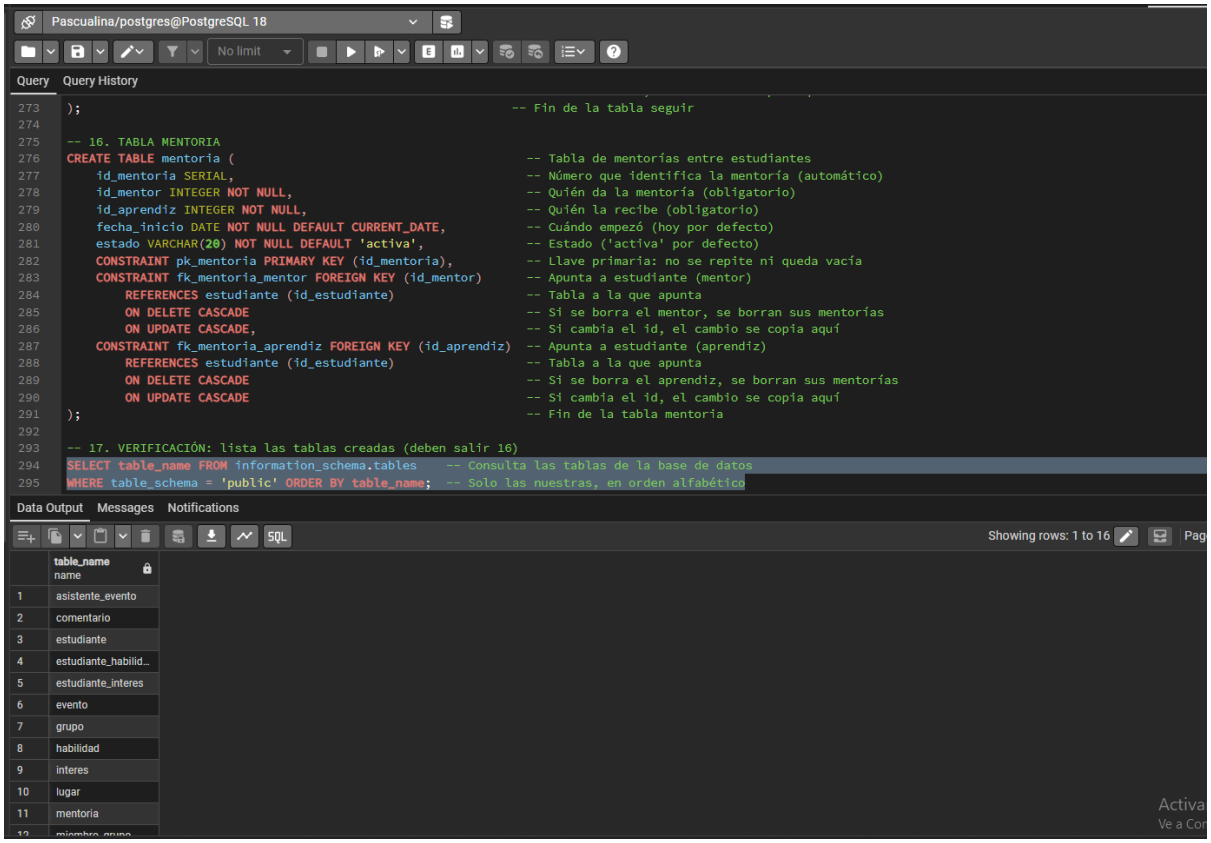
6. Diagrama y evidencias

Diagrama de las 16 tablas generado con la herramienta ERD de pgAdmin 4:



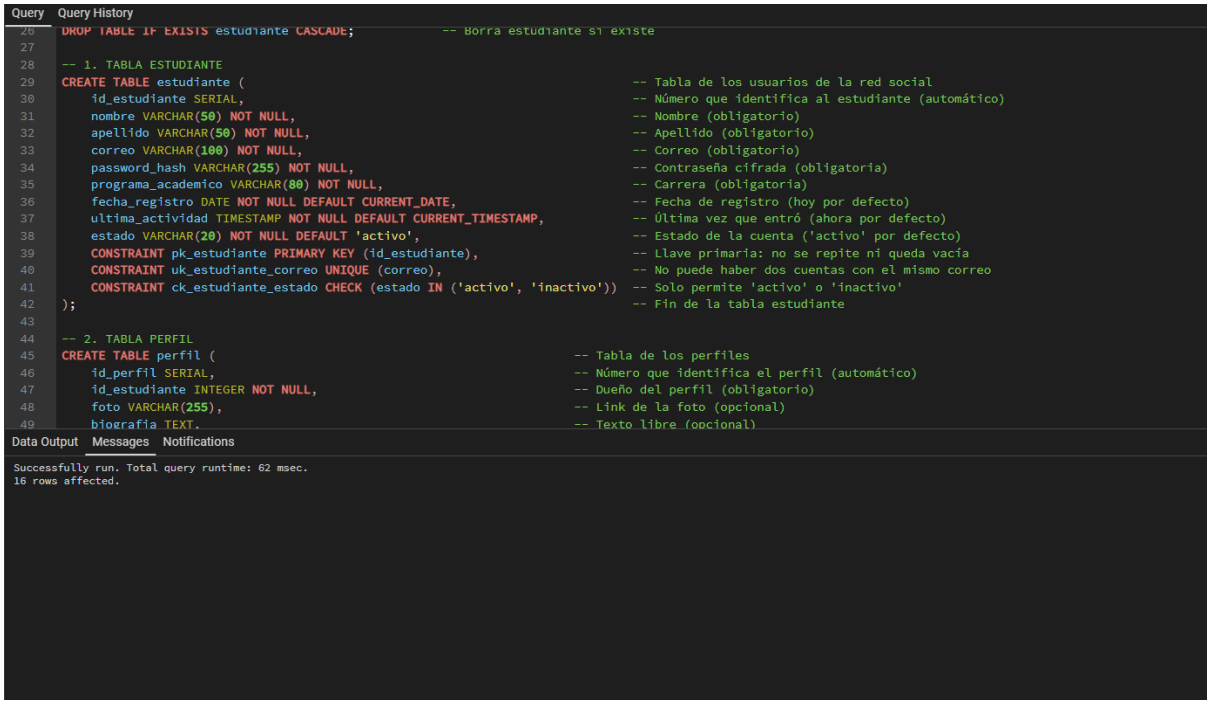
Tablas creadas

Consulta que lista las 16 tablas:



Script ejecutado

El script ejecutado en pgAdmin sin errores:



7. Conclusiones individuales

Conclusión de Andry

Esta tarea me mostró que escribir el modelo en SQL obliga a tomar decisiones que en el dibujo no se notan. Lo más difícil fue entender el ON DELETE: yo pensaba que todo se podía borrar en cascada, pero entendí que RESTRICT protege datos importantes, como un grupo con sus miembros. También me costó el orden del script, porque una tabla hija no se puede crear antes que su padre. Me queda lmirar qué reglas conviene dejar en la base de datos y cuáles en la aplicación. Como mejora, me gustaría probarlo con más datos. Creo que esto me sirve como desarrolladora, porque casi todo los sistemas guardan su información en una base de datos.

Conclusión de Alejandro

Lo que más me dejó esta tarea fue entender que escribir el DDL es tomar decisiones y no solo traducir, por ejemplo elegir VARCHAR o TEXT, o INTEGER y no BIGINT, y poder explicar por qué. Lo más complicado fue mantener las 16 tablas iguales al modelo lógico de la Tarea 2, con los mismos nombres y tipos. Me gustó que reglas que antes eran solo una idea, como que nadie entre dos veces al mismo grupo, ahora las cumple la propia base de datos. Me gustaría probar el script con datos de ejemplo desde el principio.